SmartPlayer — HEX

Documento de Estrategia Técnica

Ramón Cherta González

1. Descripción del Problema

HEX es un juego de estrategia de información perfecta para dos jugadores sobre un tablero de $N \times N$ celdas hexagonales. El Jugador 1 debe conectar el borde izquierdo con el derecho; el Jugador 2 debe conectar el borde superior con el inferior. Los jugadores alternan turnos colocando una ficha en cualquier celda vacía. La propiedad de Nash garantiza que HEX no puede terminar en empate.

Desde el punto de vista de la IA, HEX es un problema de búsqueda adversarial de suma cero. El reto principal es el tamaño del espacio de búsqueda: en un tablero 11×11 hay hasta 121! estados posibles.

2. Arquitectura del SmartPlayer

La solución se construye en cinco capas:

- Fase 1 — Navegación del tablero: vecinos even-r, detección de victoria (BFS)
- Fase 2 — Heurística Dijkstra: $h(s) = \text{dist}_{\text{opp}} - \text{dist}_{\text{self}}$
- Fase 3 — Minimax + Alpha-Beta: búsqueda adversarial con poda α-β
- Fase 4 — Filtro CSP
- Fase 5 — Control de tiempo: opening book, profundidad adaptativa, budget guard

3. Fase 1 — Fundamentos del Dominio

3.1. Sistema de coordenadas even-r

El tablero se modela con el sistema even-r offset: las filas pares están alineadas a la izquierda y las impares desplazadas media celda a la derecha. Esto produce dos conjuntos de seis direcciones según la paridad de la fila del origen:

```
# Filas pares
dirs_even = [(-1,-1),(-1,0),(0,-1),(0,1),(1,-1),(1,0)]

# Filas impares
dirs_odd  = [(-1,0),(-1,1),(0,-1),(0,1),(1,0),(1,1)]
```

3.2. Detección de victoria

La victoria no es alcanzar una celda concreta sino conectar un borde completo con otro. Se implementa como BFS multi-fuente: la cola se inicializa simultáneamente con todas las celdas del borde inicial del jugador. El objetivo es verificar si alguna celda del borde opuesto resulta alcanzable a través de fichas propias. Ventaja sobre el nodo virtual: misma complejidad $O(N^2)$ sin overhead de estructura de datos adicional

4. Fase 2 — Heurística de Evaluación (Dijkstra)

4.1. Modelo de costes

- Celda propia $\rightarrow 0$
- Celda vacía $\rightarrow 1$
- Celda rival $\rightarrow \infty$

4.2. Función de evaluación

$$h(s) = \text{dist}_{oponente} - \text{dist}_{propio}$$

- $h > 0$ ventaja propia
- $h = 0$ equilibrio
- $h < 0$ desventaja
- $+\infty$ victoria propia
- $-\infty$ derrota

5. Fase 3 — Minimax + Alpha-Beta

5.1. Árbol de juego

El árbol de juego alterna nodos MAX (turno propio) y nodos MIN (turno del oponente). La profundidad se limita para garantizar el presupuesto de 5 segundos. Al alcanzar el horizonte, se aplica la función h(s) de Fase 2.

5.2. Poda

Se mantienen dos cotas durante el recorrido:

- α(alpha): la mejor puntuación que MAX ya garantiza en el camino actual; MAX nunca aceptará un valor menor.
- β(beta): la mejor puntuación que MIN ya garantiza en el camino actual; MIN nunca aceptará un valor mayor.

```
# Nodo MAX
if value >= beta:
    break
alpha = max(alpha, value)

# Nodo MIN
if value <= alpha:
    break
beta = min(beta, value)
```

5.3. Iterative Deepening

La búsqueda se ejecuta a profundidades 1, 2, 3, ... en incrementos, guardando siempre el mejor movimiento de la última profundidad completada. Si el timer se activa a mitad de una profundidad, se retorna el resultado de la profundidad anterior, que es siempre válido y correcto.

6. Fase 4 — Filtro CSP

6.1. Restricciones

Restricción	Definición	Justificación
C1 — Adyacencia	Celdas vacías vecinas de cualquier ficha	Las fichas solo crean valor a través de la conectividad
C2 — Camino crítico	Celdas en la ruta Dijkstra de coste mínimo de cada jugador	Optimiza la conexión propia o bloquea la del oponente
C3 — Puente virtual	Vecindad común entre dos fichas del mismo color	Pivote que asegura o rompe conexiones potenciales

El conjunto candidato es:

$$C = C1 \cup C2 \cup C3$$

Si el tablero está completamente vacío (apertura), el filtro devuelve todas las celdas para no restringir el primer movimiento. Los candidatos se ordenan mejor-primero mediante h(s), maximizando los cortes de la poda Alpha-Beta. Impacto medido: en un tablero 11×11 a mitad de partida, el filtro reduce los candidatos de 101 a 56 (reducción $\approx 45\,\%$) . Con b=15 en lugar de b=100, la búsqueda Alpha-Beta a profundidad 4 pasa de 25M nodos a 50K nodos, permitiendo alcanzar profundidad 3 en menos de 0.5 segundos.

7. Fase 5 — Control de Tiempo

7.1. Opening Book

Si hay 0 ó 1 ficha en el tablero, el jugador retorna inmediatamente el centro $(N//2, N//2)$.. El centro es el movimiento más fuerte en HEX: maximiza la conectividad potencial en todas las direcciones y es equidistante de todos los bordes. Si el oponente ya ocupa el centro, se elige el vecino más cercano al eje de conexión propio.

7.2. Profundidad adaptativa

El techo de profundidad se escala inversamente con N para garantizar que el iterative deepening siempre encuentre al menos profundidad 1 antes de que expire el timer:

- $N \leq 5 \rightarrow 20$
- $N \leq 7 \rightarrow 10$
- $N \leq 9 \rightarrow 7$
- $N \leq 11 \rightarrow 5$
- $N > 11 \rightarrow 4$

7.3. Budget Guard

Para evitar iniciar una profundidad que con alta probabilidad no completará, se aplica el siguiente criterio antes de cada iteración:

```
if prev_depth_elapsed > 0.45 * remaining_time:
    break
```

Con $DEPTH_BUDGET_FRAC = 0{,}45$, si la profundidad anterior consumió más del 45se detiene la búsqueda y se retorna el mejor movimiento de la última profundidad completada.

7.4. Invariantes de seguridad

- `_TIME_GUARD = 4.3 s` (umbral interno)
- `TIME_LIMIT = 4.5 s` (SLA público)
- Límite del torneo = $5{,}0s$ (margen de 700 ms ante jitter del sistema)
- Fallback greedy garantizado: `play()` nunca retorna `None`

8. Resumen de Propiedades

Propiedad	**Valor**
Tiempo máximo	<4.35 s en todos los tamaños probados
Descalificaciones	0
Reducción candidatos	45 % especto al espacio vacío completo
Profundidad	3 en 0.5 s con CSP; 1 sin CSP
Primer movimiento	Centro N/2, N/2 (instantáneo)
Win-rate	100 % vs Random
Tamaños soportados	Cualquier N (tablero variable, sin código N-específico)
Idempotencia entre partidas	Sin estado mutable entre llamadas a play()
Dependencias	Ninguna — solo librería estándar de Python